

Universidade Federal de Uberlândia — UFU

Faculdade de Computação — FACOM



Estimativa de Conformidade a SLAs em Serviços de Vídeo sob Demanda mediante Aprendizado de Máquina

*Análise Exploratória, Regressão Linear, Classificação Logística e
Extensão 5G*

Autores: Raquel de Fátima Alves e Victor Guimarães Silva

Professor: Rafael Pasquini

PGC308A — Internet do Futuro

27 de junho de 2026

Uberlândia — MG

Resumo

Este relatório apresenta o desenvolvimento do trabalho de estimativa de conformidade com Acordos de Nível de Serviço (SLAs), realizado na disciplina Internet do Futuro — PGC308A, da Universidade Federal de Uberlândia. O estudo avalia a conformidade de um serviço de vídeo sob demanda (VoD) em relação a um SLA, utilizando um trace composto por 3.600 observações coletadas a cada segundo durante uma hora. As estatísticas do sistema operacional do servidor compõem o vetor de entrada (X), enquanto a taxa de frames exibidos no cliente (`DispFrames`) representa a métrica de qualidade (Y). Considera-se conformidade quando $\text{DispFrames} \geq 18$ fps. Na Task I, foi realizada a exploração descritiva e visual dos dados. Na Task II, aplicou-se regressão linear, obtendo-se NMAE de 10,39% no conjunto de teste (1.080 observações), abaixo do limiar de 15%. Na Task III, utilizou-se regressão logística para classificar a conformidade ao SLA, alcançando o melhor ERR de 11,11% (C5). Como atividade bônus, foi implementada a predição do indicador `Qual` em traces 5G, obtendo MAE de 3,20 dB e $R^2 = 0,29$. O código-fonte completo está disponível em <https://github.com/victorguimaraessilva/Trabalho-IA-MSc>.

Palavras-chave: SLA; QoS; vídeo sob demanda; regressão linear; regressão logística; garantia de serviço; redes 5G; aprendizado de máquina.

Sumário

1	Introdução	2
2	Materiais e Métodos	2
2.1	Cenário e base de dados VoD	2
2.2	Protocolo de validação	3
3	Task I — Exploração dos Dados	4
3.1	Item 1 — Estatísticas descritivas	4
3.2	Item 2 — Consultas específicas	6
3.3	Item 3 — Visualizações	6
4	Task II — Estimativa da Métrica de Serviço (Regressão)	7
4.1	Item 1 — Modelo M com treino completo	7
4.1.1	Itens 1(c-e) — Gráficos no conjunto de teste	9
4.2	Item 2 — Impacto do tamanho do treino ($M_1 \dots M_5$)	10
5	Task III — Estimativa de Conformidade ao SLA (Classificação)	11
5.1	Item 1 — Desempenho dos classificadores $C_1 \dots C_5$	11

6	Atividade Bônus — 5G Datasets Challenge (Atividade A)	13
6.1	Descrição	13
6.2	Resultados	13
7	Conclusão	14

1 Introdução

Serviços de telecomunicações e aplicações de Internet dependem de infraestruturas complexas, nas quais a qualidade percebida pelo usuário deve ser continuamente monitorada e garantida. Nesse contexto, os Acordos de Nível de Serviço (*Service Level Agreements* — SLAs) formalizam compromissos de desempenho entre provedor e cliente [?]. Neste trabalho, a qualidade de um serviço de vídeo sob demanda (VoD) é avaliada a partir da taxa de frames exibidos no terminal do cliente, enquanto o provedor dispõe apenas de métricas coletadas no servidor em que o serviço é hospedado.

A hipótese investigada é que relações estatísticas entre métricas de infraestrutura e qualidade percebida no cliente podem ser aprendidas por modelos supervisionados de aprendizado de máquina [?]. Dessa forma, torna-se possível estimar, em tempo quase real, tanto o valor contínuo da métrica de serviço (regressão) quanto a conformidade binária ao SLA (classificação). O enunciado propõe o uso de modelos lineares interpretáveis como linha de base, priorizando a compreensão dos coeficientes e da relação entre as variáveis em vez da busca pela máxima acurácia preditiva.

O relatório está organizado conforme a ordem das tarefas propostas no enunciado: Task I (exploração), Task II (regressão linear), Task III (classificação logística) e atividade bônus (traces 5G). Em cada seção, são apresentados a metodologia adotada, os resultados, as figuras e tabelas correspondentes, e a interpretação dos principais achados.

2 Materiais e Métodos

2.1 Cenário e base de dados VoD

Utilizou-se o trace público *realm-in 2015-vod-traces* [?] (3.600 observações, amostragem 1 Hz). Os arquivos `X.csv` e `Y.csv` foram unidos pelo campo `TimeStamp`. A configuração conceitual consiste em um cliente conectado a um servidor VoD via rede; X captura o estado do SO no servidor e Y a taxa de frames no cliente. A Tabela 1 descreve todas as variáveis.

Tabela 1: Variáveis do trace VoD.

Identificador	Descrição
<code>all_..idle</code>	Percentual de tempo em que a CPU do servidor está ociosa (%)
<code>X..memused</code>	Percentual de memória RAM utilizada no servidor (%)
<code>proc.s</code>	Taxa de criação de processos no sistema (processos/s)
<code>cswch.s</code>	Taxa de troca de contexto da CPU (trocas/s)
<code>file.nr</code>	Quantidade de file handles abertos no sistema operacional
<code>sum_intr.s</code>	Taxa de interrupções de hardware e software (interrupções/s)
<code>ldavg.1</code>	Média de carga da CPU nos últimos 60 segundos
<code>tcpsck</code>	Número de sockets TCP atualmente em uso
<code>pgfree.s</code>	Taxa de liberação de páginas de memória (páginas/s)
<code>DispFrames</code>	Taxa de frames de vídeo exibidos no cliente (fps) — alvo Y

2.2 Protocolo de validação

Para as Tasks II e III, adotou-se a técnica de conjunto de validação (*validation set*): 2.520 observações para treino (70%) e 1.080 para teste (30%), com amostragem aleatória uniforme e `random_state=42` para reprodutibilidade [?]. A partir do conjunto de treino, geraram-se subconjuntos de 50, 500, 1.000, 1.500 e 2.520 observações para estudar o impacto do tamanho da amostra de treino.

A implementação foi realizada em Python 3, com `pandas` (manipulação de dados), `scikit-learn` (modelos), `matplotlib` e `seaborn` (visualização). O código está organizado em notebooks por tarefa e módulos reutilizáveis em `src/`. O trecho a seguir ilustra as constantes centrais:

Listing 1: Constantes centrais do projeto (`src/config.py`).

```

1 SLA_THRESHOLD_FPS = 18      # fps -- limiar de conformidade ao
   SLA
2 TRAIN_FRACTION      = 0.70  # 70% treino / 30% teste
3 RANDOM_STATE        = 42    # semente para reprodutibilidade
4 TRAIN_SIZES = [50, 500, 1000, 1500, 2520]
```

3 Task I — Exploração dos Dados

3.1 Item 1 — Estatísticas descritivas

O Item 1 solicita, para cada componente de X e Y , as estatísticas: média, máximo, mínimo, percentil 25, percentil 90 e desvio padrão. A Tabela 2 consolida esses valores, calculados sobre as 3.600 observações do trace.

Tabela 2: Estatísticas descritivas de X e Y (3.600 observações).

Estatística	CPU ociosa (%)	Memória usada (%)	Taxa proc. (/s)	Troca ctx. (/s)	File handles	Interrup. (/s)	Carga CPU (1 min)	Sockets TCP	Lib. pág. (/s)	DispFrames (fps)
Média	9,0650	89,1375	7,6833	54045,87	2656,33	19978,04	75,8758	48,9975	72872,15	18,8184
Máximo	69,5400	97,8400	48,0000	83880,00	2976,00	35536,00	147,4700	87,0000	145874,00	30,3900
Mínimo	0,0000	73,0300	0,0000	11398,00	2304,00	10393,00	11,1300	21,0000	15928,00	0,0000
Percentil 25	0,0000	82,9650	0,0000	31302,00	2496,00	16678,00	28,2000	34,0000	61601,75	13,3900
Percentil 90	38,6210	96,7700	20,0000	72135,10	2880,00	28228,40	127,9930	71,0000	97532,50	24,6100
Desvio padrão	16,1228	8,1837	8,5326	19497,81	196,11	4797,27	43,8624	15,8712	19504,32	5,2198

Interpretação: a taxa média de frames foi de 18,82 fps, valor muito próximo ao limiar de SLA (18 fps), indicando que o serviço opera frequentemente na fronteira entre conformidade e degradação. A memória média ($\approx 89\%$) e a baixa CPU ociosa ($\approx 9\%$) sugerem servidor sob carga sustentada durante o trace. O desvio padrão de `DispFrames` (5,22 fps) revela variabilidade significativa na qualidade entregue ao cliente.

3.2 Item 2 — Consultas específicas

Tabela 3: Respostas às consultas do Item 2.

Item	Resultado	Interpretação
(a)	2.875 observações	Quantidade de amostras com uso de memória superior a 80%
(b)	46,3473 sockets	Média de sockets TCP quando a taxa de interrupções excede 18.000/s
(c)	73,03%	Menor utilização de memória entre amostras com CPU ociosa abaixo de 20%

Em 2.875 das 3.600 observações (79,9%), a memória superou 80%, reforçando a pressão de recursos. Mesmo com CPU pouco ociosa, a memória mínima permaneceu em 73,03%, ou seja, o servidor não apresentou folga significativa de RAM sequer nos momentos de maior utilização de CPU.

O código a seguir ilustra como foram obtidas as respostas do Item 2:

Listing 2: Consultas específicas sobre o dataset (Item 2 da Task I).

```

1 # (a) observacoes com memoria > 80%
2 q2a = int((df["X..memused"] > 80).sum()) # resultado: 2875
3
4 # (b) media de sockets TCP quando interrupcoes > 18000/s
5 q2b = df.loc[df["sum_intr.s"] > 18000, "tcpsck"].mean() #
   46.3473
6
7 # (c) minimo de memoria quando CPU ociosa < 20%
8 q2c = df.loc[df["all_..idle"] < 20, "X..memused"].min() #
   73.03

```

3.3 Item 3 — Visualizações

Item 3(a): série temporal de CPU ociosa e memória usada.

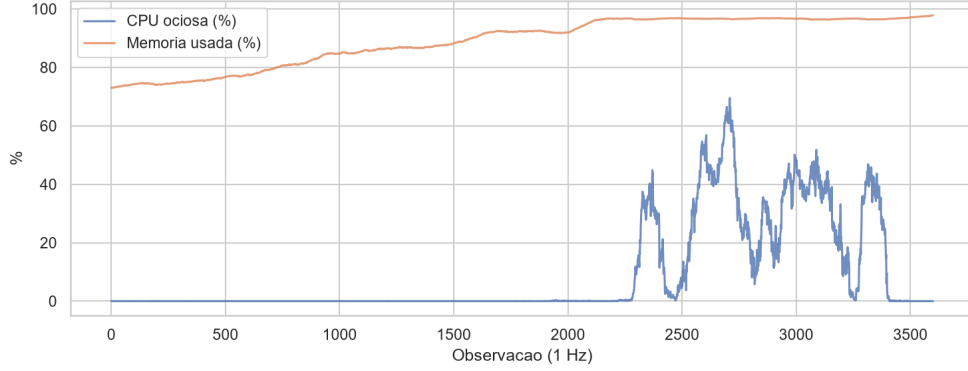


Figura 1: Evolução temporal da CPU ociosa e da memória utilizada ao longo das 3.600 observações.

A série temporal (Figura 1) revela variabilidade ao longo da hora de coleta, com memória persistentemente elevada e picos de utilização de CPU. A CPU ociosa permanece próxima de zero na maior parte do tempo, com surtos pontuais que podem refletir períodos de menor demanda pelo serviço VoD.

Item 3(b): estimativas de densidade kernel (KDE) para CPU ociosa e memória.

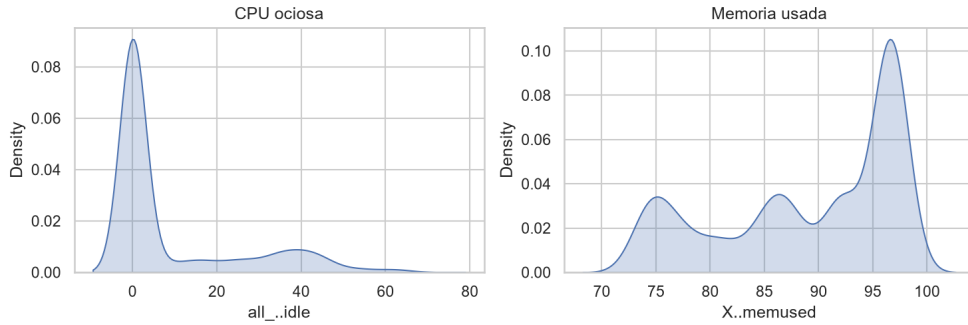


Figura 2: Distribuição (KDE) da CPU ociosa e da memória utilizada.

As curvas KDE (Figura 2) complementam as estatísticas descritivas: a memória concentra-se em faixas altas (80–97%), enquanto a CPU ociosa apresenta distribuição assimétrica com moda próxima de zero. Esse padrão é consistente com um servidor operando próximo ao limite de capacidade de forma sustentada.

4 Task II — Estimativa da Métrica de Serviço (Regressão)

4.1 Item 1 — Modelo M com treino completo

O objetivo é aprender uma função $M : X \rightarrow \hat{Y}$ que aproxime `DispFrames`. Utilizou-se regressão linear múltipla (`LinearRegression` do `scikit-learn`). O erro de estima-

ção é o NMAE:

$$\text{NMAE} = \frac{1}{\bar{y}} \cdot \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i| \quad (1)$$

onde $\bar{y}$ é a média de `DispFrames` no conjunto de teste e $m = 1.080$. O modelo é considerado preciso se $\text{NMAE} < 15\%$.

O trecho abaixo mostra o treinamento e avaliação do modelo M:

Listing 3: Treinamento e avaliação do modelo de regressão linear (Task II).

```

1  from sklearn.linear_model import LinearRegression
2  from src.metrics import nmae
3  from src.splits import train_test_split_trace
4
5  train_df, test_df, X_train, y_train, X_test, y_test = \
6      train_test_split_trace(df, features, target)
7
8  model = LinearRegression()
9  model.fit(X_train, y_train)
10 y_pred = model.predict(X_test)
11
12 erro = nmae(y_test, y_pred)  # normaliza pelo y_bar do teste

```

Tabela 4: Coeficientes do modelo M ($\Theta_1 \dots \Theta_9$ e Θ_0).

Variável	Coeficiente
CPU ociosa (%)	−0,0921
Memória usada (%)	−0,0868
Taxa de processos (/s)	−0,0120
Troca de contexto (/s)	−0,0001
File handles	−0,0031
Interrupções (/s)	+0,0000
Carga CPU (1 min)	−0,0606
Sockets TCP	−0,0653
Liberação de páginas (/s)	−0,0000
Intercepto (Θ_0)	49,6878

Tabela 5: Desempenho do modelo M no conjunto de teste.

Métrica	Valor	Critério
NMAE	10,3877%	< 15%
Acurado?	Sim	—
Observações de teste	1.080	—

O NMAE de 10,39% indica erro médio relativo modesto. Coeficientes negativos em variáveis de pressão do servidor (memória, carga, sockets TCP) associam-se a estimativas menores de fps, coerente com a degradação de serviço sob estresse de recursos. O intercepto de 49,69 representa o valor base do modelo quando todas as variáveis são nulas — situação hipotética sem interpretação física direta.

4.1.1 Itens 1(c–e) — Gráficos no conjunto de teste

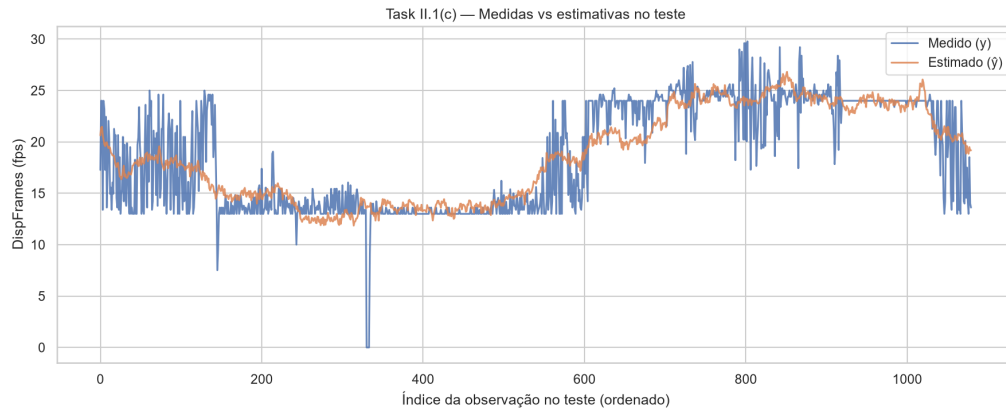
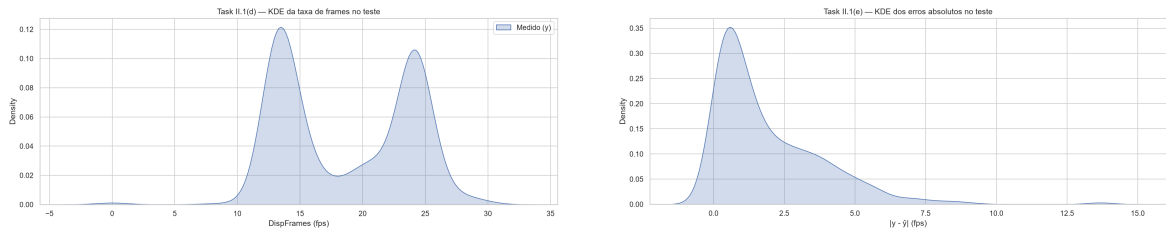


Figura 3: Valores medidos e estimados de `DispFrames` no conjunto de teste.

O modelo acompanha a tendência geral dos dados (Figura 3), com divergências nos extremos e em transições rápidas. Nos períodos em que `DispFrames` cai abruptamente para valores próximos de zero, o modelo não consegue capturar a queda com a mesma velocidade, o que é esperado em um modelo linear sem componente temporal.



(a) KDE de `DispFrames` no teste.

(b) KDE dos erros absolutos $|y - \hat{y}|$.

Figura 4: Distribuições de `DispFrames` e dos erros absolutos no conjunto de teste (Task II).

A Figura 4a revela distribuição bimodal de **DispFrames** no teste, com concentrações em torno de 14 fps e 24 fps, refletindo dois regimes de operação do servidor. O KDE dos erros absolutos (Figura 4b) mostra que a maioria dos erros se concentra abaixo de 5 fps, com cauda longa para eventos extremos.

Item 1(f) — Discussão: a regressão linear oferece baseline interpretável e atende ao critério de acurácia. Limitações incluem a incapacidade de capturar não-linearidades e a sensibilidade a outliers na taxa de frames.

4.2 Item 2 — Impacto do tamanho do treino ($M_1 \dots M_5$)

Tabela 6: NMAE e tempo de treino para diferentes tamanhos de conjunto de treino.

Treino (obs.)	NMAE (%)	Tempo (ms)	Acurado?
50	11,52	1,13	Sim
500	10,58	1,86	Sim
1.000	10,49	1,19	Sim
1.500	10,38	1,22	Sim
2.520	10,39	2,59	Sim

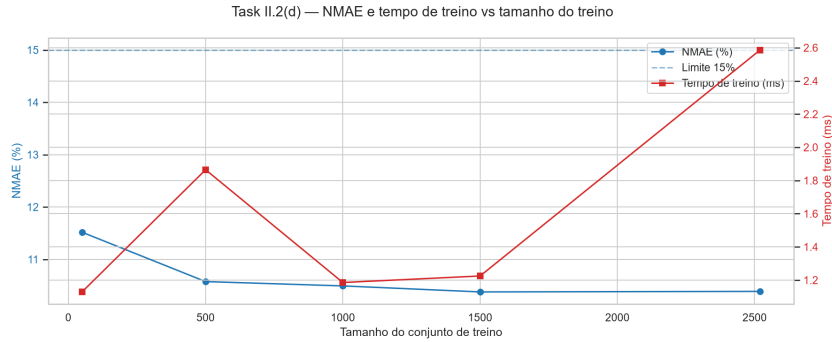


Figura 5: NMAE (%) e tempo de treino versus tamanho do conjunto de treino (Task II).

Item 2(e): com apenas 50 observações o NMAE já é aceitável ($\approx 11,5\%$); ganhos após 500 amostras são marginais — caracterizando **retornos decrescentes** (Figura 5).

Item 2(f): o tempo de treino permanece abaixo de 3 ms, insignificante frente ao volume de dados disponível, o que reforça a viabilidade de re-treinamento frequente em ambientes de monitoramento em tempo real.

5 Task III — Estimativa de Conformidade ao SLA (Classificação)

Define-se rótulo 1 (conforme) se $\text{DispFrames} \geq 18$ fps e rótulo 0 (violação) caso contrário. A métrica de avaliação é o ERR:

$$\text{ERR} = 1 - \frac{VP + VN}{m} \quad (2)$$

onde VP e VN são verdadeiros positivos e negativos e $m = 1.080$. Classificador preciso se $\text{ERR} < 15\%$.

O trecho a seguir mostra como os rótulos são criados e o classificador treinado:

Listing 4: Criação de rótulos e treinamento do classificador logístico (Task III).

```
1 from sklearn.linear_model import LogisticRegression
2 from src.metrics import sla_labels, classification_error
3
4 # binariza Y: 1 (conforme) se DispFrames >= 18, 0 (violacao)
   caso contrario
5 y_bin_train = sla_labels(y_reg_train, SLA_THRESHOLD_FPS)
6 y_bin_test  = sla_labels(y_reg_test,  SLA_THRESHOLD_FPS)
7
8 clf = LogisticRegression(max_iter=10000, random_state=
   RANDOM_STATE)
9 clf.fit(X_train, y_bin_train)
10 y_pred = clf.predict(X_test)
11
12 err = classification_error(y_bin_test, y_pred)
```

5.1 Item 1 — Desempenho dos classificadores $C_1 \dots C_5$

Tabela 7: Desempenho dos classificadores $C_1 \dots C_5$ no conjunto de teste.

Clf.	Treino	Tempo (ms)	ERR (%)	Acurado?	VP	VN	FP	FN
C1	50	118,95	15,37	Não	438	476	59	107
C2	500	275,09	12,96	Sim	465	475	60	80
C3	1.000	262,29	12,87	Sim	464	477	58	81
C4	1.500	1.373,22	11,39	Sim	479	478	57	66
C5	2.520	2.526,55	11,11	Sim	486	474	61	59

O melhor classificador foi C5, com $ERR = 11,11\%$. Com apenas 50 amostras de treino, $ERR = 15,37\%$, ligeiramente acima do limite. Os falsos negativos (FN) — instâncias em violação classificadas como conformes — caem de 107 (C1) para 59 (C5), o que é crítico do ponto de vista operacional: FN levam o provedor a não agir em situações de degradação real.

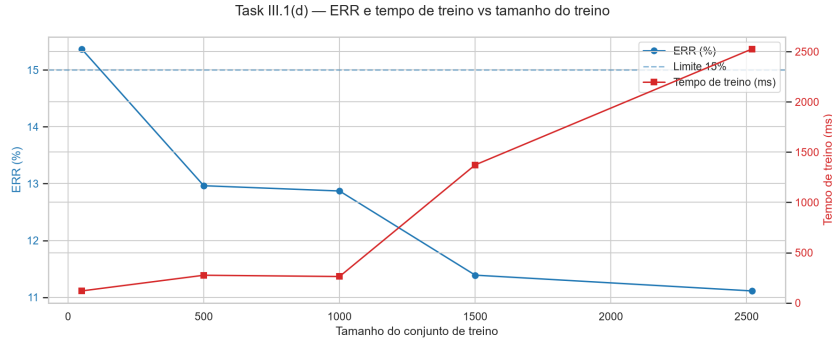


Figura 6: ERR (%) e tempo de treino versus tamanho do conjunto de treino (Task III).

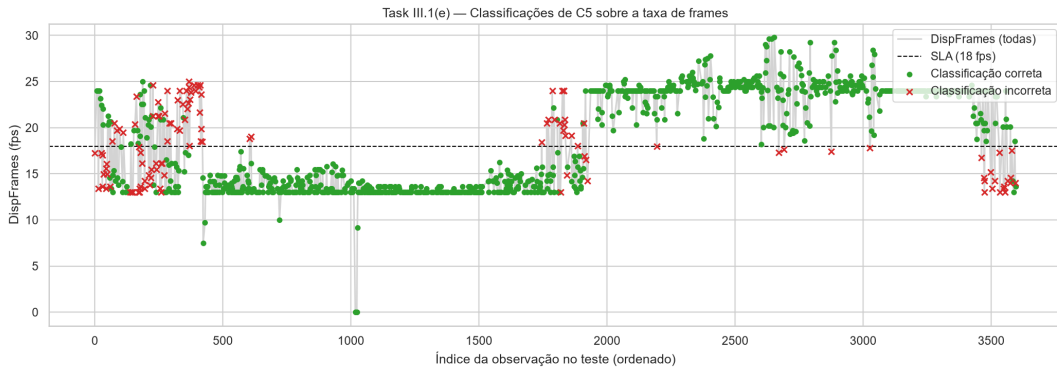


Figura 7: Série temporal de **DispFrames** no teste com classificações de C5 (verde = acerto, vermelho = erro).

Item 1(f): erros concentram-se próximo a 18 fps — fronteira entre classes (Figura 7). Longe do limiar, o modelo classifica corretamente com alta consistência.

Item 1(g): mais dados de treino reduzem ERR de forma consistente (Figura 6), com ganho expressivo entre 50 e 500 amostras e retornos decrescentes a partir daí.

Item 1(h): o tempo de treino cresce com o dataset (de ≈ 119 ms a ≈ 2.527 ms), mas permanece prático para uso em produção ou re-treinamento periódico.

6 Atividade Bônus — 5G Datasets Challenge (Atividade A)

6.1 Descrição

Como atividade opcional, foi implementada a **Atividade A** do 5G Datasets Challenge (SMARTNESS/UNICAMP): predição de qualidade de sinal (**Qual**/RSRQ, em dB) a partir de métricas de rede coletadas com Samsung S21 5G na rede Claro em São Paulo.

Foram utilizados três traces do dataset **g-nettrack-pro**:

Tabela 8: Traces utilizados na Atividade Bônus.

Arquivo	Cenário
2023-01-21_308-walking-paulista-1.txt	Pedestre — Av. Paulista
2023-01-22_933-driving-sp-1.txt	Veículo em deslocamento
2023-01-21_244-subway-1.txt	Metrô

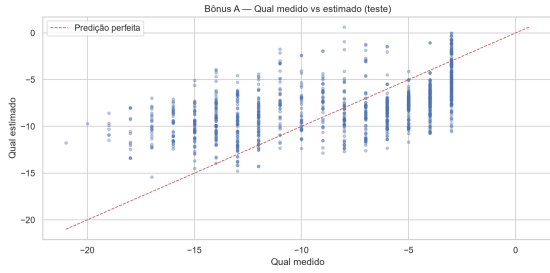
As features utilizadas foram: **Level** (RSRP, dBm), **SNR**, **DL_bitrate** (kbps), **UL_bitrate** (kbps) e **Speed** (km/h). Após remoção de valores inválidos (–, –99), restaram ≈ 4.500 registros. O mesmo protocolo de split 70/30 com **random_state=42** foi aplicado.

6.2 Resultados

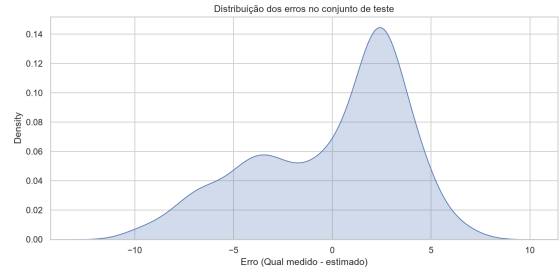
Tabela 9: Métricas do modelo de regressão linear para predição de **Qual** (5G).

Métrica	Valor	Interpretação
MAE	3,20 dB	Erro médio absoluto entre Qual medido e estimado
RMSE	3,80 dB	Raiz do EQM — penaliza erros grandes
R^2	0,29	Fração da variância de Qual explicada pelo modelo

O coeficiente de determinação $R^2 = 0,29$ indica relação parcial — esperado para um modelo linear simples aplicado a dados heterogêneos que combinam cenários de pedestre, veículo e metrô (indoor/outdoor), com eventos de handover não modelados. A variável **SNR** é a de maior influência ($\Theta = +0,346$): maior relação sinal-ruído está associada a melhor qualidade de sinal estimada.



(a) Qual medido vs. estimado.



(b) KDE dos erros de estimativa.

Figura 8: Avaliação do modelo de predição de qualidade de sinal 5G (Atividade Bônus).

O scatter plot (Figura 8a) mostra dispersão considerável em torno da diagonal, confirmando R^2 moderado. O KDE dos erros (Figura 8b) apresenta distribuição aproximadamente simétrica centrada em zero, sem viés sistemático de superestimação ou subestimação — o modelo erra de forma equilibrada, o que é desejável.

7 Conclusão

Este trabalho demonstrou a viabilidade de usar modelos lineares supervisionados para estimar conformidade a SLAs em serviços VoD a partir de métricas do servidor.

Na Task II, a regressão linear atingiu NMAE de **10,39%** (abaixo do limite de 15%), com retornos decrescentes a partir de 500 amostras de treino. Na Task III, o melhor classificador logístico (C5) obteve ERR de **11,11%**, com os erros concentrados na fronteira de 18 fps — região inerentemente ambígua.

As principais limitações são: (i) a linearidade dos modelos, que pode não capturar relações não lineares; (ii) o split aleatório, que pode misturar padrões temporais; e (iii) a ausência de features de rede e do terminal do cliente.

Como trabalho futuro, propõe-se o uso de modelos não lineares (Random Forest, XGBoost), validação temporal e enriquecimento das features com métricas de rede.

Na atividade bônus, a mesma metodologia aplicada a traces 5G obteve $R^2 = 0,29$, resultado modesto mas interpretável para um baseline linear com dados de campo heterogêneos.

O código-fonte completo, notebooks reproduzíveis e dados estão disponíveis em: <https://github.com/victorguimaraessilva/Trabalho-IA-MSc>